



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Ingegneria dell'Informazione, Elettronica e Matematica

ACADEMY MEDICAL WEARABLE DEVICES

A.A. 2025/2026

Documento del Design di Dettaglio

Project BabyGuard

Il monitoraggio intelligente del sonno neonatale (IoMT)

GRUPPO 10

Bonavita Anna Pia (Mat 0612800373)

Guercia Giovanni (Mat. 0612800536)

Rossino Giacomo (Mat. 0612708792)

Tassino Antonio (Mat. 0612800290)

Indice

1. Architettura del sistema	4
1.1 Descrizione dei Moduli	4
1.1.1 Modulo Smart Shirt (Howdy Senior)	4
1.1.2 Modulo Sensore di Temperatura NTC	4
1.1.3 Modulo Gateway Edge (ESP32 / MicroPython)	4
1.1.4 Modulo BLE	5
1.1.5 Modulo MQTT Publisher	5
1.1.6 Modulo Node-RED Flow	5
1.1.7 Modulo InfluxDB (Time-Series DB)	5
1.1.8 Modulo Identity & Records Management (DB Relazionale via SQLAlchemy)	6
1.1.9 Modulo API REST (FastAPI Controllers)	6
1.1.10 Modulo Analitica Clinica — Calcolo Indice AHI	6
1.1.11 Modulo Grafana	7
1.1.12 Modulo App React Native	7
1.1.13 Modulo Notifiche	7
1.2 Implementazione dei componenti	7
1.2.1 Flusso dati ad alte prestazioni (serie temporali)	7
1.2.2 Interazione con l'Identity & Records Management	7
1.2.3 Flusso analitico cross-database: calcolo dell'Indice AHI	8
1.3 Pattern Architetture e di Design	8
1.3.1 Polyglot Persistence	8
1.3.2 Publish-Subscribe (MQTT)	8
1.3.3 Edge Sensor Fusion	9
2. Modello statico	10
2.1 Principi di Buona Progettazione	10
2.1.1 Principi Generali	10
2.1.2 Principi SOLID	10
2.1.3 Principio di Robustezza	11
2.2 Coesione	12
2.3 Accoppiamento	12
2.4 Modello dei Dati e Persistenza (Database Design)	13
2.4.1 Database Relazionale (SQLite) e Schema ER	13
2.4.1.1 Diagramma ER (Modello Concettuale)	13
2.4.1.2 Tabelle Relazionali (Modello Logico)	13
2.4.1.3 Dizionario dei Dati	14
2.4.2 Database Temporale (InfluxDB) e Schema della Telemetria	15
2.4.3 Data Security e Retention Policy	15
3. Modello dinamico e Design dell'interfaccia	16
3.1 Diagrammi di sequenza	16
3.1.1 Acquisizione Parametri Vitali	16
3.1.2 Generazione di un Allarme Critico	16
3.1.3 Calcolo e consultazione dell'Indice AHI	17
3.2 Diagrammi delle attività	18

3.2.1 Acquisizione Parametri Vitali	18
3.2.2 Generazione di un Allarme Critico	19
3.2.3 Calcolo dell'Indice AHI	20
4. Design dell'interfaccia utente	22
4.1 Schermata di Login	22
4.2 Dashboard Real-time Genitore	22
4.3 Area Pazienti Pediatra	23
Appendice A — Tracciabilità delle modifiche verso i requisiti	24

1. Architettura del sistema

L'architettura del sistema BabyGuard è progettata secondo un approccio modulare e stratificato, finalizzato a garantire manutenibilità, scalabilità, sicurezza dei dati clinici e una chiara separazione delle responsabilità. Il sistema è strutturato su quattro livelli — wearable/edge, comunicazione, backend/middleware e frontend mobile — che permettono di ottimizzare le prestazioni del flusso dati continuo e, al tempo stesso, di isolare la gestione dell'identità e dei permessi dalla gestione delle rilevazioni fisiologiche. La revisione corrente non altera la stratificazione complessiva, ma ne arricchisce il livello edge con una sorgente di misura locale (NTC) e il livello backend con una funzione analitica (Indice AHI).

1.1 Descrizione dei Moduli

Di seguito sono descritti i moduli che compongono i quattro livelli architetturali, con le rispettive responsabilità, dipendenze e note di revisione.

Livello 1 — Wearable / Edge

1.1.1 Modulo Smart Shirt (Howdy Senior)

Responsabilità	Funge da interfaccia con il corpo del bambino. I sensori tessili integrati acquisiscono i parametri multi-modalità: IMU (accelerometro/giroscopio) per l'orientamento spaziale e l'agitazione motoria; Strain Gauge (fascia di respiro) per gli atti respiratori e il rilevamento delle apnee; sensore ECG per l'estrazione della frequenza cardiaca. I dati vengono inviati al gateway tramite BLE.
Dipendenze	Modulo Gateway Edge (ESP32) — destinatario dei dati BLE.

1.1.2 Modulo Sensore di Temperatura NTC

Responsabilità	Termistore NTC collegato direttamente all'ESP32 sul lato edge. Misura la temperatura cutanea del neonato come grandezza analogica letta dal convertitore ADC del microcontrollore. È la nuova ed unica sorgente del parametro di temperatura, in sostituzione del sensore tessile della smart shirt.
Caratteristiche	Lettura locale a bassa latenza, indipendente dallo stato del collegamento BLE; il valore grezzo viene linearizzato/convertito in °C dal firmware dell'ESP32 prima della pubblicazione.
Dipendenze	Modulo Gateway Edge (ESP32) — host fisico del sensore e responsabile della conversione e validazione del campione.

1.1.3 Modulo Gateway Edge (ESP32 / MicroPython)

Responsabilità	Riceve via BLE i dati multi-modalità (ECG, Strain Gauge, IMU) dalla smart shirt e, contestualmente, acquisisce in locale la temperatura cutanea dal sensore NTC ad esso collegato, linearizza il valore e lo unisce ai dati multi-modalità ricevuti dalla maglietta. Pubblica i dati aggregati via MQTT in tempo reale verso il backend, delegando ad
-----------------------	---

	esso l'applicazione dei filtri di stabilizzazione, la valutazione delle soglie cliniche e l'innesco degli allarmi.
Dipendenze	Modulo Smart Shirt (dati multi-modalità via BLE); Modulo Sensore NTC (temperatura cutanea locale); Modulo MQTT Publisher (pubblicazione verso il broker); Configurazione delle soglie (ricevuta dal backend).

Livello 2 — Comunicazione

1.1.4 Modulo BLE

Responsabilità	Gestisce la comunicazione a bassissimo consumo tra la smart shirt e l'ESP32, in modo da non gravare sulla batteria del wearable, garantendo un canale locale affidabile per il trasferimento continuo delle rilevazioni multi-modalità.
Dipendenze	Modulo Smart Shirt e Modulo Gateway Edge (estremi del canale).

1.1.5 Modulo MQTT Publisher

Responsabilità	Sfrutta la connessione Wi-Fi dell'ESP32 per pubblicare i pacchetti JSON sul broker, garantendo un trasferimento leggero e continuo verso il server secondo il modello publish-subscribe.
Dipendenze	Modulo Gateway Edge (sorgente dei messaggi); Modulo Node-RED Flow (subscriber a valle del broker).

Livello 3 — Backend e Middleware (Data & Identity Management)

Il cuore del sistema è orchestrato tramite Docker/Docker Compose e adotta un'architettura polyglot persistence, con due database dai compiti distinti.

1.1.6 Modulo Node-RED Flow

Responsabilità	Funge da middleware instradando i flussi MQTT. Trasforma e pulisce i dati provenienti dai dispositivi prima di scriverli nel database temporale, normalizzando il formato delle rilevazioni.
Dipendenze	Modulo MQTT Publisher (sorgente dei messaggi); Modulo InfluxDB (destinazione delle rilevazioni).

1.1.7 Modulo InfluxDB (Time-Series DB)

Responsabilità	Memorizza esclusivamente le rilevazioni dei sensori e l'ID del dispositivo, garantendo altissime prestazioni in scrittura per i dati continui. Non contiene dati anagrafici o identificativi del paziente.
Dipendenze	Riceve scritture dal Modulo Node-RED Flow; Viene interrogato dal Modulo API REST, dal Modulo Analitica Clinica (AHI) e dal Modulo Grafana.

1.1.8 Modulo Identity & Records Management (DB Relazionale via SQLAlchemy)

Responsabilità	Gestisce l'anagrafica, le credenziali cifrate e i ruoli (Genitore/Pediatra). Associa in modo sicuro l'ID del dispositivo al genitore e al medico curante, custodisce i permessi di accesso e conserva le soglie critiche configurate.
Dipendenze	Modulo API REST (verifica credenziali, permessi e lettura dello storico allarmi/soglie).

1.1.9 Modulo API REST (FastAPI Controllers)

Responsabilità	Interfaccia sicura che espone i dati all'applicazione mobile ed ospita il modulo di monitoraggio in tempo reale (<code>`mqtt_handler.py`</code>). Questo modulo ascolta i dati dai topic MQTT, applica i filtri digitali (EMA sui battiti, varianza sull'accelerazione per l'ipotonia), verifica il superamento delle soglie cliniche (lette dal database SQLite) e genera gli allarmi, inviandoli via WebSocket ed associandoli a Telegram. Costituisce l'unico punto di accesso controllato ai dati clinici per il frontend.
Dipendenze	Modulo Identity & Records Management (autenticazione, permessi, storico allarmi); Modulo InfluxDB (estrazione delle rilevazioni e delle ore di monitoraggio).

1.1.10 Modulo Analitica Clinica — Calcolo Indice AHI

Responsabilità	Componente analitico del livello backend (parte dei FastAPI Controllers) che calcola l' Indice di Apnea-Ipopnea (AHI) come rapporto tra il numero di eventi respiratori anomali (apnee/SIDS registrate) e le ore di sonno/monitoraggio del neonato. Restituisce il valore dell'indice e la relativa classe di severità.
Logica di calcolo	Numeratore: conteggio degli allarmi Apnea/SIDS dal DB relazionale nella finestra richiesta; Denominatore: ore di monitoraggio stimate da InfluxDB contando i campioni di temperatura (cadenza ~1 ogni 5 s $\Rightarrow$ 720 campioni = 1 ora); $AHI = \text{eventi} / \text{ore}$, con guardia contro la divisione per zero quando le ore sono trascurabili.
Classificazione	Normale ($AHI < 5$) · Lieve (< 15) · Moderato (< 30) · Severo (≥ 30). La lettura va contestualizzata sull'età postmestruale, poiché l'AHI neonatale è fisiologicamente elevato e decresce con la maturazione.
Dipendenze	Modulo Identity & Records Management (storico allarmi);

	Modulo InfluxDB (ore di monitoraggio attivo).
--	---

1.1.11 Modulo Grafana

Responsabilità	Piattaforma di data visualization connessa direttamente a InfluxDB che consente al pediatra di analizzare le dashboard cliniche dal proprio computer per la consultazione storica dei parametri, inclusa la serie temporale della temperatura cutanea.
Dipendenze	Modulo InfluxDB (sorgente delle serie temporali).

Livello 4 — Frontend Mobile

1.1.12 Modulo App React Native

Responsabilità	Applicazione multiplatforma (iOS/Android) che si connette al backend tramite chiamate API REST. L'interfaccia è suddivisa per ruolo: il genitore visualizza solo il dispositivo associato al figlio e ne monitora lo stato in tempo reale; il pediatra accede alla lista dei propri pazienti, potendo selezionare un bambino per consultarne report, storico essenziale e Indice AHI.
Dipendenze	Modulo API REST (accesso ai dati e all'indice AHI); Modulo Notifiche (ricezione degli allarmi).

1.1.13 Modulo Notifiche

Responsabilità	Gestisce la presentazione delle notifiche push all'utente autorizzato (allarme clinico, batteria scarica, disconnessione del dispositivo).
Dipendenze	Modulo API REST / Backend (sorgente degli eventi di notifica).

1.2 Implementazione dei componenti

L'implementazione dei componenti distingue tre tipologie di flusso: il flusso ad alte prestazioni delle serie temporali, il flusso di gestione dell'identità e degli accessi e il flusso analitico cross-database per il calcolo dell'Indice AHI.

1.2.1 Flusso dati ad alte prestazioni (serie temporali)

Il percorso delle rilevazioni è ottimizzato per la continuità e il volume. La smart shirt produce i dati multi-modalità (ECG, Strain Gauge, IMU); il gateway ESP32 li unisce alla temperatura cutanea letta localmente dal sensore NTC, aggrega e filtra l'insieme e ne pubblica i pacchetti via MQTT. Node-RED, in qualità di subscriber, normalizza i messaggi ed scrive su InfluxDB. Contestualmente, il backend FastAPI riceve i dati MQTT e li instrada in tempo reale tramite connessione WebSocket attiva direttamente all'applicazione mobile del genitore, aggiornando i tracciati della dashboard a bassissima latenza.

1.2.2 Interazione con l'Identity & Records Management

Il Modulo API REST (FastAPI) agisce da mediatore tra il frontend e i due database. Quando l'applicazione mobile richiede dati clinici, il controller verifica innanzitutto il token di sessione e i permessi nel Modulo Identity & Records Management. Solo dopo aver accertato che

l'utente è autorizzato a vedere uno specifico dispositivo, il controller risolve l'associazione utente–Device ID e interroga InfluxDB per estrarre le rilevazioni corrispondenti. In questo modo l'identità del paziente e le sue misure restano fisicamente separate e vengono ricongiunte solo al momento dell'accesso autorizzato.

1.2.3 Flusso analitico cross-database: calcolo dell'Indice AHI

Il calcolo dell'AHI è l'esempio paradigmatico di **ricongiungimento applicativo dei due store** della polyglot persistence. Su richiesta autenticata dell'app, il controller dedicato esegue tre passi: (1) conta nello storico allarmi del DB relazionale gli eventi di apnea/SIDS nella finestra temporale richiesta (numeratore); (2) stima da InfluxDB le ore di monitoraggio attivo contando i campioni di temperatura della serie `vitals_summary` (denominatore); (3) calcola l'indice come rapporto eventi/ore, ne arrotonda il valore e ne deriva la classe di severità. La separazione fisica dei dati è preservata: il join avviene a livello applicativo, in memoria, esclusivamente per il paziente autorizzato.

Passo	Sorgente	Operazione	Esito
1. Eventi anomali	DB relazionale (storico allarmi)	Conteggio degli allarmi Apnea/SIDS nella finestra richiesta	apnea_count (numeratore)
2. Ore di monitoraggio	InfluxDB (vitals_summary.temperatura)	Conteggio campioni di temperatura ÷ 720 (≈1 campione/5 s)	hours (denominatore)
3. Indice e severità	Livello applicativo	AHI = apnea_count / hours; classificazione di severità	ahi_index + status

1.3 Pattern Architetture e di Design

1.3.1 Polyglot Persistence

Il sistema adotta la Polyglot Persistence impiegando due database eterogenei, ciascuno scelto in base alla natura dei dati: InfluxDB per le serie temporali ad alta frequenza e un database relazionale (SQLite/PostgreSQL via SQLAlchemy) per l'anagrafica, i permessi, le soglie e lo storico allarmi. Questa scelta coniuga le prestazioni di scrittura di un Time-Series DB sul flusso clinico con l'integrità referenziale e la sicurezza di un database relazionale sui dati identificativi, realizzando una separazione strutturale tra dato clinico e dato personale. **Il calcolo dell'Indice AHI valorizza appieno questo pattern:** l'indice nasce dal join applicativo tra il conteggio degli eventi (relazionale) e le ore di monitoraggio (time-series), senza che i due store debbano conoscersi reciprocamente.

1.3.2 Publish-Subscribe (MQTT)

La trasmissione dei dati dall'ESP32 al backend adotta il pattern Publish-Subscribe tramite MQTT. L'ESP32 pubblica i pacchetti su specifici topic del broker senza conoscere i consumatori, mentre Node-RED vi si sottoscrive. **Il dato di temperatura NTC viaggia su un topic dedicato**, coerentemente con la natura disaccoppiata del pattern: l'aggiunta della sorgente locale non ha richiesto modifiche ai consumatori, che si limitano a sottoscrivere il nuovo topic. Il disaccoppiamento consente di aggiungere o rimuovere dispositivi e logiche di

elaborazione senza modificare gli altri componenti, gestendo più dispositivi concorrenti in modo scalabile.

1.3.3 Edge Sensor Fusion

Il gateway ESP32 realizza un pattern di **sensor fusion sull'edge**: integra in un unico modello di dato due sorgenti eterogenee per provenienza e trasporto — i parametri multi-modalità ricevuti via BLE dalla smart shirt e la temperatura cutanea letta in locale dall'NTC tramite ADC. La fusione avviene prima della pubblicazione, così che il livello a valle riceva un pacchetto omogeneo e indipendente dalla topologia fisica dei sensori. Il pattern accresce la robustezza, perché la misura termica resta disponibile anche in presenza di disturbi sul canale BLE.

2. Modello statico

2.1 Principi di Buona Progettazione

La progettazione di BabyGuard è condotta con l'obiettivo di garantire manutenibilità, scalabilità e una complessità contenuta. Sono stati applicati i seguenti principi generali e orientati agli oggetti, qui aggiornati per riflettere l'acquisizione locale via NTC e il calcolo dell'Indice AHI.

2.1.1 Principi Generali

Principio	Definizione	Applicazione nel progetto
Separazione delle preoccupazioni (SoC)	I diversi aspetti del sistema sono separati in livelli e moduli indipendenti.	La logica di acquisizione edge (ESP32 + NTC), l'instradamento (Node-RED), la persistenza temporale (InfluxDB), l'identità e lo storico (DB relazionale), l'analitica clinica (calcolo AHI) e la presentazione (React Native) sono nettamente distinte. La sorgente NTC è un dettaglio di acquisizione confinato all'edge; il calcolo dell'AHI è confinato al backend.
KISS (Keep It Simple, Stupid)	Si predilige una struttura modulare chiara, evitando complessità non necessarie.	Il gateway ESP32 espone una logica lineare (acquisisci BLE + NTC – filtra – valuta soglie – pubblica); l'AHI è calcolato con una semplice formula eventi/ore esposta da un unico endpoint.
YAGNI (You Aren't Going to Need It)	Non vengono introdotte funzionalità non richieste.	Su InfluxDB si salvano esclusivamente le rilevazioni e il Device ID; le ore di monitoraggio per l'AHI sono derivate da un campo già presente (temperatura) anziché aggiungere contatori dedicati.
DRY (Don't Repeat Yourself)	Ogni logica ha un'unica astrazione, evitando ripetizioni.	La validazione delle soglie è centralizzata; l'accesso ai dati clinici e il calcolo dell'AHI passano esclusivamente dai FastAPI Controllers, evitando duplicazioni della logica di autorizzazione e di aggregazione.

2.1.2 Principi SOLID

Principio	Definizione	Applicazione nel progetto
Single Responsibility (SRP)	Ogni modulo svolge un solo compito ben definito.	L'Identity & Records Management gestisce identità, permessi e storico; InfluxDB gestisce le rilevazioni; il calcolo

Principio	Definizione	Applicazione nel progetto
		dell'AHI è una responsabilità analitica distinta, separata dal monitoraggio real-time. L'acquisizione della temperatura è responsabilità dell'edge (NTC), non più della smart shirt.
Open-Closed (OCP)	I moduli sono chiusi alla modifica ma aperti all'estensione.	L'introduzione della sorgente NTC e dell'indice AHI è avvenuta per estensione: un nuovo topic MQTT e un nuovo endpoint, senza riscrivere il gateway, Node-RED o i moduli esistenti.
Liskov Substitution (LSP)	Gli oggetti devono poter essere sostituiti dai loro sottotipi senza alterare la correttezza.	Il DB relazionale è acceduto tramite l'astrazione SQLAlchemy: SQLite o PostgreSQL sono intercambiabili senza modifiche alla logica applicativa, inclusa la query sullo storico allarmi usata dall'AHI.
Interface Segregation (ISP)	Un client non deve dipendere da metodi che non utilizza.	L'app del genitore espone gli endpoint di monitoraggio real-time del proprio dispositivo; il pediatra dispone di interfacce dedicate a lista pazienti, storico, dashboard Grafana e indice AHI. Nessun client dipende da funzionalità estranee al proprio ruolo.
Dependency Inversion (DIP)	I moduli di alto livello dipendono da astrazioni, non dai dettagli.	I FastAPI Controllers (alto livello), anche nel calcolo dell'AHI, interagiscono con i dati tramite astrazioni (client InfluxDB e ORM SQLAlchemy): un cambiamento dei dettagli di persistenza non si propaga alla logica analitica o di controllo.

2.1.3 Principio di Robustezza

Il sistema è progettato per “limitare i danni” quando le precondizioni non sono rispettate, evitando comportamenti ambigui e perdite di dati.

- **Filtraggio dei rumori di lettura:** l'ESP32 aggrega le letture BLE e la lettura NTC, scartando i valori fisiologicamente non plausibili o transitori, così che artefatti di misura non generino allarmi falsi o scritture inutili.
- **Misura termica resiliente:** essendo l'NTC locale al gateway, la temperatura resta disponibile anche durante disturbi o brevi interruzioni del canale BLE, aumentando la robustezza del parametro termico.
- **Convalida del superamento delle soglie:** prima dell'invio in rete, il gateway verifica il superamento delle soglie critiche integrando il dato NTC; solo gli eventi significativi vengono pubblicati come allarme, riducendo traffico e falsi positivi.

- **Guardia nel calcolo dell'AHI:** il backend evita la divisione per zero quando le ore di monitoraggio sono trascurabili, restituendo un indice neutro anziché un valore indefinito.
- **Gestione degli errori a valle:** Node-RED e i FastAPI Controllers intercettano le anomalie (messaggi malformati, token non valido, paziente non autorizzato, neonato privo di dispositivo) restituendo errori controllati senza compromettere la continuità del servizio.

2.2 Coesione

La progettazione mira a un'alta coesione interna ai moduli. La tabella riepiloga i livelli di coesione e la forma di accoppiamento prevalente per le principali tipologie di componenti.

Modulo	Livello di coesione	Accoppiamento prevalente
Smart Shirt	Comunicazionale — gli elementi operano sugli stessi dati (segnali fisiologici) e cooperano alla loro acquisizione.	Per Dati — scambia con il gateway solo i valori delle rilevazioni multi-modalità via BLE.
Sensore NTC (edge)	Funzionale — unico scopo: fornire la temperatura cutanea locale all'ESP32.	Per Dati — espone all'ESP32 il solo campione termico (grandezza analogica).
Gateway Edge (ESP32)	Funzionale — fonde BLE e NTC, filtra e valida, producendo pacchetti pronti alla pubblicazione.	Per Dati — comunica a valle solo tramite i pacchetti JSON pubblicati, senza controllarne il flusso di esecuzione.
Node-RED Flow	Logica — raggruppa operazioni differenti ma collegate (instradamento, trasformazione, pulizia) sui messaggi.	Per Dati — riceve e produce esclusivamente messaggi/rilevazioni.
API REST (FastAPI)	Comunicazionale — i controller operano sugli stessi dati (richieste clinico-anagrafiche) coordinando i due database.	Per Controllo — decidono quale logica invocare (verifica permessi, estrazione dati, calcolo AHI).
Analitica Clinica (AHI)	Funzionale — unico scopo: derivare l'indice eventi/ore e la severità.	Per Dati — riceve conteggio allarmi e ore di monitoraggio e restituisce un valore aggregato.
Identity & Records Management	Funzionale — autenticazione, ruoli, associazioni, soglie e storico allarmi.	Per Dati — espone esiti di verifica e record (ruolo, permessi, conteggi) come valori di ritorno.

2.3 Accoppiamento

Le scelte progettuali privilegiano un basso accoppiamento tra i moduli, in linea con la stratificazione architeturale.

- **Accoppiamento per Dati:** forma prevalente. Lo scambio tra livelli avviene tramite messaggi e parametri (pacchetti JSON via MQTT, payload delle API REST), senza che un modulo controlli il flusso di esecuzione di un altro.
- **Accoppiamento per Timbro:** si manifesta dove un componente conosce la struttura completa di un'entità; ad esempio i FastAPI Controllers, per autorizzare l'accesso e per calcolare l'AHl, accedono alla struttura dell'associazione utente–dispositivo e ai record dello storico allarmi gestiti dall'Identity & Records Management.

L'acquisizione locale via NTC **riduce ulteriormente l'accoppiamento esterno** del parametro termico: la temperatura non dipende più dal collegamento BLE con la smart shirt ma da un sensore interno al gateway. Grazie al pattern publish-subscribe, inoltre, l'accoppiamento tra ESP32 e backend è minimo: produttore e consumatore comunicano attraverso il broker, ottenendo la massima indipendenza tra livello edge e middleware.

2.4 Modello dei Dati e Persistenza (Database Design)

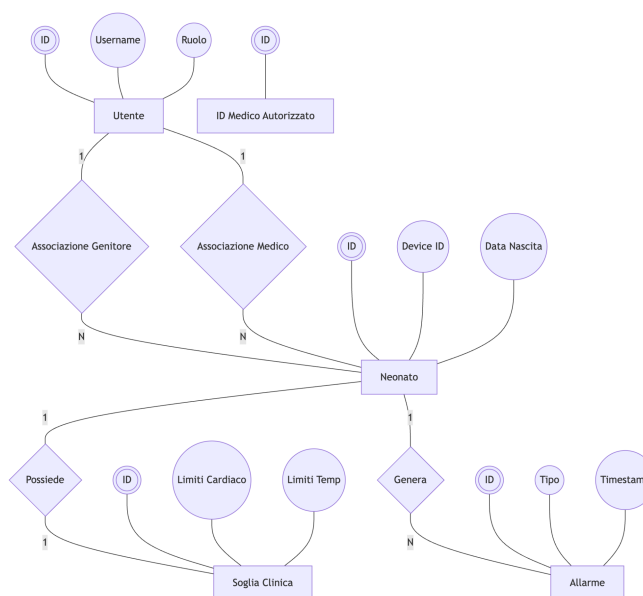
In linea con il pattern architetturale - Polyglot Persistence - descritto nella sezione 1.3.1, la persistenza del sistema BabyGuard è divisa tra un database relazionale SQLite per i dati strutturati e un database temporale InfluxDB per la telemetria continua ad alta frequenza.

2.4.1 Database Relazionale (SQLite) e Schema ER

Il database relazionale SQLite viene utilizzato per la gestione di identità, permessi, impostazioni dei pazienti, storico degli allarmi e autorizzazioni mediche. La progettazione è suddivisa in un modello concettuale (Diagramma ER) ed un modello logico.

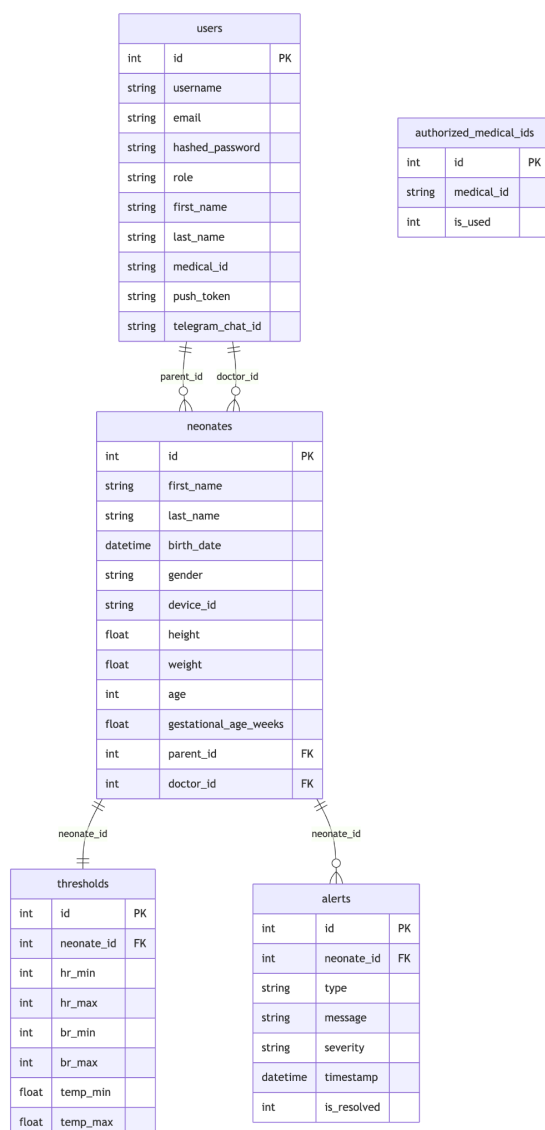
2.4.1.1 Diagramma ER (Modello Concettuale)

Il diagramma Entità-Relazione (ER) concettuale rappresenta le entità del dominio, i loro attributi principali e le relazioni logiche astratte, senza scendere nei dettagli implementativi o fisici del database.



2.4.1.2 Tabelle Relazionali (Modello Logico)

Il modello logico traduce il modello concettuale ER in una struttura a tabelle relazionali idonea per SQLite. In questo modello vengono esplicitate le chiavi primarie (PK), le chiavi esterne (FK) e i vincoli di unicità.



2.4.1.3 Dizionario dei Dati

- ``users`` (Utenti del Sistema): Memorizza tutti gli utenti registrati (Pediatri, Genitori, Amministratori). Contiene credenziali, ruoli e i token per le notifiche (Telegram Chat ID per notifiche asincrone e push token per il dispositivo mobile).
- ``neonates`` (Pazienti Neonati): Rappresenta i neonati associati. Il campo ``device_id`` (univoco) funge da chiave di accoppiamento logico con le misure memorizzate nel database delle serie temporali InfluxDB. Gli attributi ``parent_id`` e ``doctor_id`` fungono da chiavi esterne referenziate alla tabella ``users``.
- ``thresholds`` (Soglie Cliniche Personalizzate): Contiene le soglie cliniche minime e massime impostate dal pediatra per i parametri di Heart Rate, Breath Rate e Temperatura di ogni singolo neonato. Ha una relazione 1-a-1 con ``neonates``.
- ``alerts`` (Storico degli Allarmi): Tabella in cui vengono persistiti tutti gli allarmi generati dal backend (``type`` = HR, BR, Temp, Position, Battery) con marcatura temporale e lo stato di risoluzione (``is_resolved``), garantendo la tracciabilità per l'anamnesi.
- ``authorized_medical_ids`` (Codici Medici Autorizzati): Registro di sicurezza contenente gli identificativi dei medici autorizzati ad iscriversi. Il flag ``is_used`` impedisce l'uso duplicato dello stesso codice.

2.4.2 Database Temporale (InfluxDB) e Schema della Telemetria

I dati biometrici in tempo reale inviati dall'ESP32 vengono memorizzati nel database di serie temporali InfluxDB, nel bucket `babyguard\_bucket`.

La tabella `neonates` su SQLite si collega a InfluxDB associando il campo `device\_id` al tag indicizzato `shirt\_id`.

I dati su InfluxDB sono organizzati in tre Measurement principali:

- 1) `vitals\_summary` (Sintesi Parametri Vitali):
 - Tag indicizzato: `shirt\_id` (identificativo maglietta).
 - Fields: `temperature` (float, °C), `heartrate` (float/int, bpm), `breathrate` (float/int, atti al minuto) e `status` (int, stato sensore).
- 2) `biometric\_waves` (Onde Biometriche ad Alta Frequenza):
 - Tags indicizzati: `shirt\_id`, `data\_type` (es. `ACC\_GYRO` per accelerometro/giroscopio).
 - Fields: `samples` (stringa JSON contenente le letture ad alta frequenza dell'accelerometro per il calcolo della varianza del movimento e della postura del neonato).
- 3) `device\_diagnostics` (Diagnostica Hardware):
 - Tag indicizzato: `shirt\_id`.
 - Fields: `battery\_soc` (int, stato di carica della batteria da 0% a 100%).

2.4.3 Data Security e Retention Policy

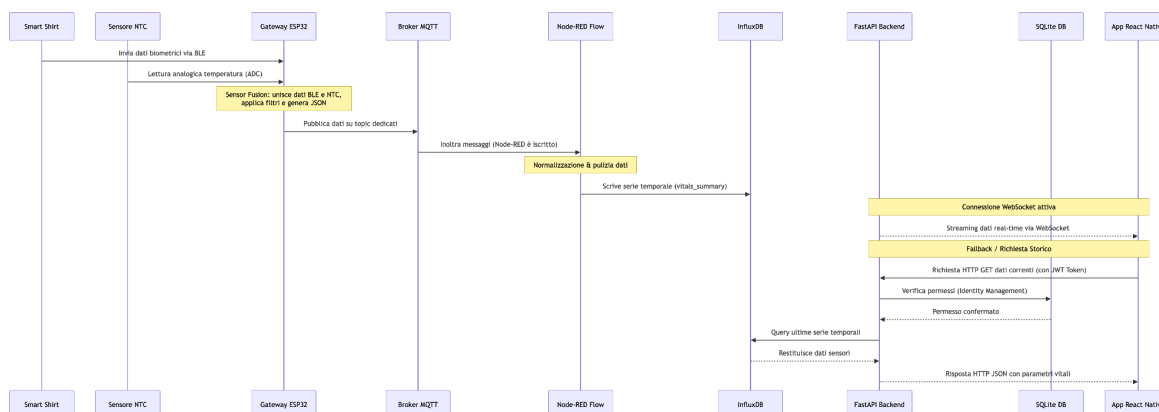
- Sicurezza delle credenziali: Le password degli utenti nella tabella `users` vengono cifrate a livello applicativo tramite algoritmo di hashing robusto bcrypt prima del salvataggio nel database relazionale.
- Autenticazione delle API: L'accesso alle informazioni sensibili è protetto tramite token JWT (JSON Web Token) con firma digitale HMACS256 generata dal backend.
- Retention Policy delle serie temporali: Per ottimizzare lo spazio su disco ed evitare saturazione dovuta alla telemetria ad alta frequenza, sul bucket InfluxDB è impostata una Retention Policy automatica di 30 giorni. I dati strutturati (anagrafica, allarmi clinici) su SQLite vengono invece persistiti senza limiti temporali per scopi clinici storici.

3. Modello dinamico e Design dell'interfaccia

3.1 Diagrammi di sequenza

3.1.1 Acquisizione Parametri Vitali

Descrizione del diagramma: questo diagramma di sequenza descrive il flusso con cui un parametro vitale viene acquisito e reso disponibile, in tempo reale, sull'applicazione del genitore. I partecipanti sono: Smart Shirt, Sensore NTC, Gateway Edge (ESP32), Broker MQTT, Node-RED, InfluxDB, API REST (FastAPI), Identity & Records Management e App React Native (Genitore).



Flusso logico:

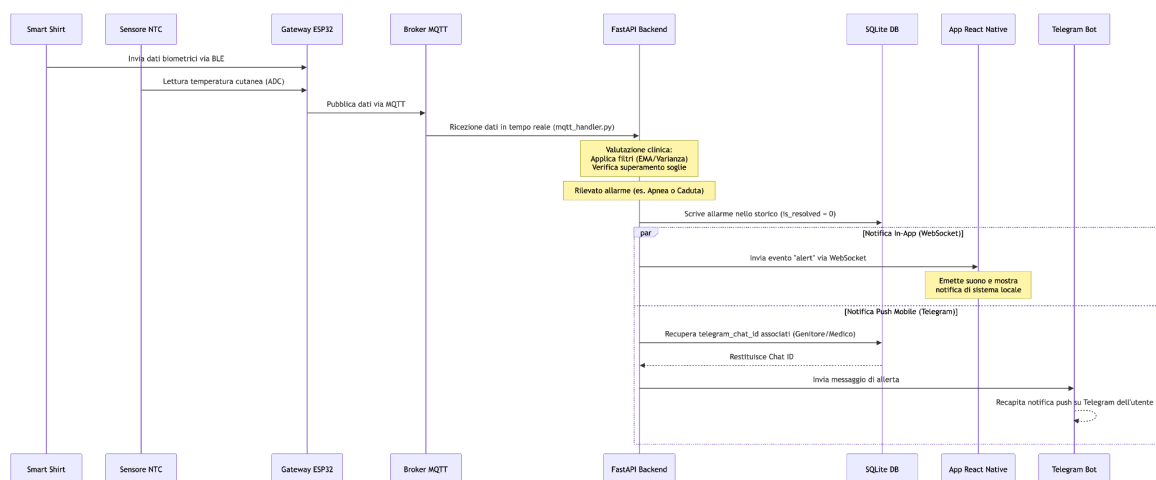
1. La Smart Shirt acquisisce i parametri multi-modalità (ECG, Strain Gauge, IMU) e li invia via BLE al Gateway Edge.
2. Il Sensore NTC fornisce al Gateway Edge la temperatura cutanea come lettura analogica locale (nessun passaggio via BLE).
3. Il Gateway Edge **fonde le due sorgenti**, filtra i rumori e costruisce un pacchetto JSON con le rilevazioni (incluso il valore NTC) e il Device ID.
4. Il Gateway Edge pubblica il pacchetto sul Broker MQTT (la temperatura sul topic dedicato).
5. Node-RED, sottoscritto ai topic, riceve i messaggi, li trasforma e li normalizza.
6. Node-RED scrive le rilevazioni su InfluxDB, indicizzate per tempo e Device ID.
7. L'App React Native del genitore richiede i dati correnti alle API REST, includendo il token di sessione.
8. Le API REST verificano il token e i permessi presso l'Identity & Records Management.
9. Una volta autorizzate, le API REST risolvono l'associazione utente–Device ID ed estraggono le ultime rilevazioni da InfluxDB.
Le API REST restituiscono i dati all'App, che aggiorna la dashboard real-time.

Flusso alternativo: se il token risulta non valido o scaduto, le API REST restituiscono un errore di autorizzazione e l'App reindirizza l'utente alla schermata di Login.

3.1.2 Generazione di un Allarme Critico

Descrizione del diagramma: descrive la generazione e la propagazione di un allarme critico (posizione prona persistente, apnea/SIDS o soglia termica superata), evidenziando la collaborazione tra livello edge e livello backend. I partecipanti sono: Smart Shirt, Sensore

NTC, Gateway Edge (ESP32), Broker MQTT, Node-RED, InfluxDB, Backend/API REST, Identity & Records Management, Modulo Notifiche e App React Native (Genitore).



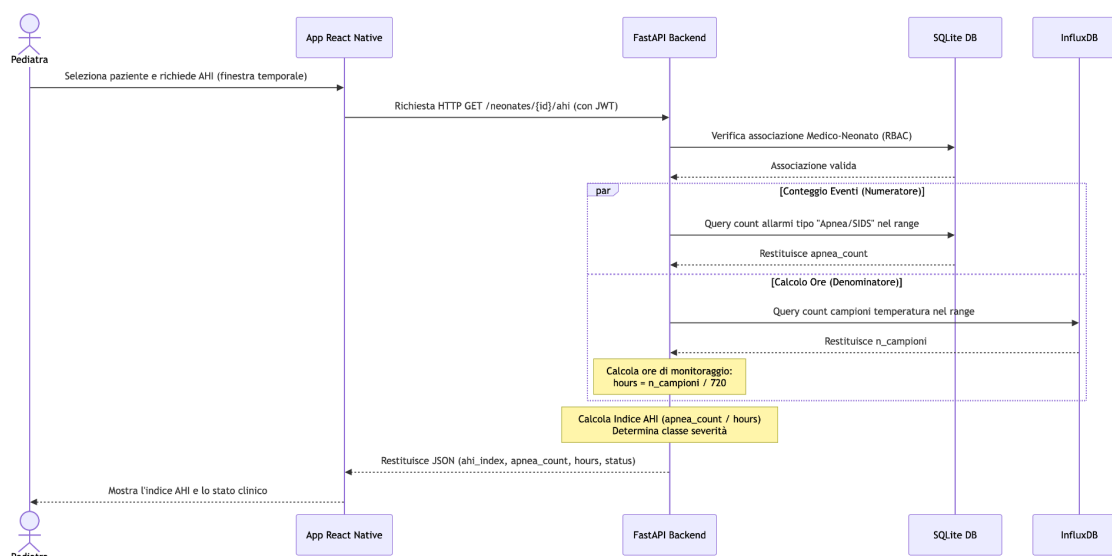
Flusso logico:

1. La Smart Shirt invia via BLE i parametri multi-modali al Gateway Edge; **in parallelo il Sensore NTC fornisce la temperatura cutanea locale.**
2. Il Gateway Edge acquisisce la temperatura dal sensore NTC, applica il filtro di linearizzazione locale e lo unisce ai dati biometrici per la pubblicazione su MQTT.
3. Il Backend FastAPI (tramite `mqtt_handler.py`) intercetta i dati MQTT, applica i filtri digitali (EMA sulla frequenza cardiaca, varianza sull'accelerazione per l'ipotonia) e confronta i valori con le soglie cliniche personalizzate memorizzate nel database SQLite.
4. Il Backend rileva il superamento di una soglia clinica o un'anomalia (es. ipertermia/ipotermia, apnea, caduta, posizione prona), innesca l'allarme e lo persiste nello storico degli allarmi (DB relazionale).
5. Node-RED riceve l'evento, lo elabora e lo persiste; l'allarme è registrato nello storico (DB relazionale) ed eventualmente marcato sulle serie InfluxDB.
6. Il Backend genera una notifica push e, tramite l'Identity & Records Management, individua il genitore associato al Device ID.
7. Il Modulo Notifiche consegna la notifica all'App React Native del genitore.
8. Il genitore riceve l'allarme e può aprire la dashboard per verificare lo stato del bambino.

Flussi alternativi: (a) se la lettura è rumorosa o transitoria, il Gateway Edge la scarta e non genera alcun allarme; (b) se il genitore è momentaneamente offline, la notifica viene riproposta alla riconnessione.

3.1.3 Calcolo e consultazione dell'Indice AHI

Descrizione del diagramma: descrive il calcolo on-demand dell'Indice AHI e la sua consultazione da parte del pediatra. I partecipanti sono: App React Native (Pediatra), API REST (FastAPI) con il modulo Analitica Clinica, Identity & Records Management e InfluxDB.



Flusso logico:

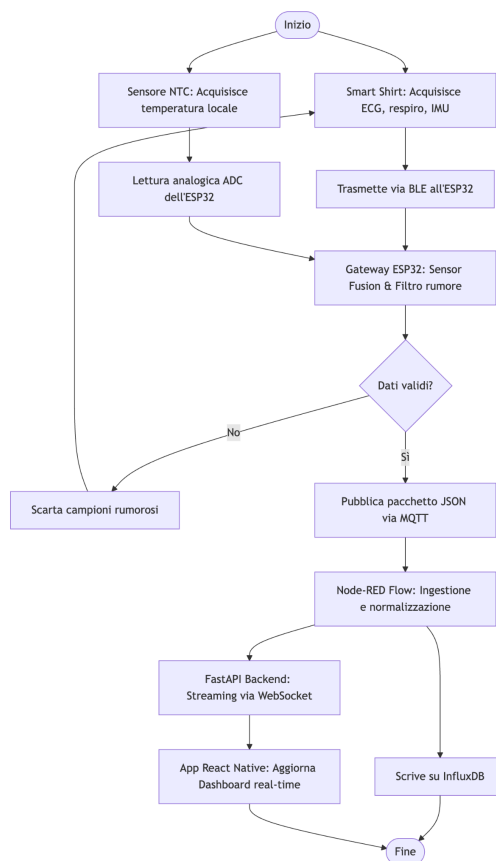
1. Il Pediatra, dall'area clinica del paziente selezionato, richiede l'Indice AHI per una finestra temporale (es. ultime 24 ore).
2. L'API REST verifica token e permessi presso l'Identity & Records Management e controlla l'associazione del paziente al pediatra.
3. Il modulo Analitica Clinica conta nello storico allarmi del DB relazionale gli eventi di apnea/SIDS nella finestra (numeratore).
4. Il modulo interroga InfluxDB e stima le ore di monitoraggio attivo dai campioni di temperatura (denominatore).
5. Calcola AHI = eventi / ore, applicando la guardia contro la divisione per zero, e ne deriva la classe di severità.
6. L'API REST restituisce indice, conteggio eventi, ore e severità; l'App li presenta nell'area clinica.

Flussi alternativi: (a) paziente non associato al pediatra → accesso negato; (b) neonato privo di dispositivo o ore di monitoraggio trascurabili → indice neutro con stato informativo.

3.2 Diagrammi delle attività

3.2.1 Acquisizione Parametri Vitali

Descrizione: il diagramma di attività illustra il flusso di esecuzione dell'acquisizione, suddiviso tra le corsie (swimlane) dei componenti coinvolti.



Attori / Corsie:

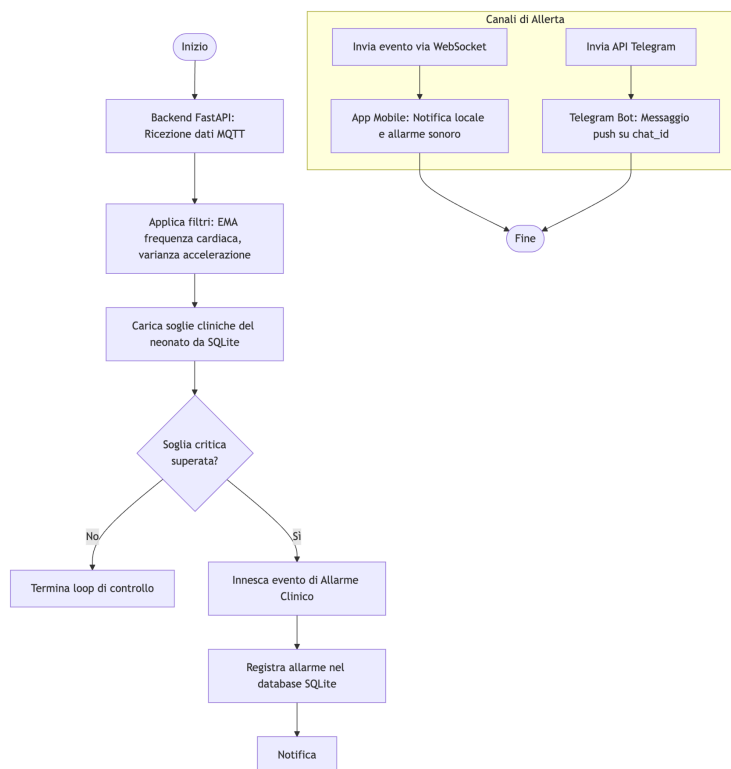
- **Smart Shirt:** acquisisce ECG, Strain Gauge e IMU e li trasmette via BLE.
- **Sensore NTC:** fornisce in locale la temperatura cutanea all'ESP32.
- **Gateway Edge (ESP32):** fonde BLE e NTC, filtra e impacchetta i dati, quindi pubblica via MQTT.
- **Node-RED:** trasforma e scrive su InfluxDB.
- **Backend/App:** espone e visualizza i dati.

Flusso degli eventi:

1. Inizio: la Smart Shirt acquisisce i parametri multi-modali e, **in parallelo, l'ESP32 legge la temperatura dal sensore NTC.**
2. Decisione di qualità: il Gateway valuta se le letture (BLE e NTC) sono valide. Se una lettura è rumorosa, viene scartata e il flusso ritorna all'acquisizione.
3. Se le letture sono valide, il Gateway fonde le sorgenti, costruisce il pacchetto JSON e lo pubblica via MQTT.
4. Node-RED normalizza e scrive la rilevazione su InfluxDB.
5. Su richiesta autorizzata dell'App, il Backend estrae il dato e l'App aggiorna la dashboard. Fine.

3.2.2 Generazione di un Allarme Critico

Descrizione: il diagramma di attività illustra il processo decisionale che porta dall'acquisizione di un parametro alla consegna della notifica di allarme.



Attori / Corsie:

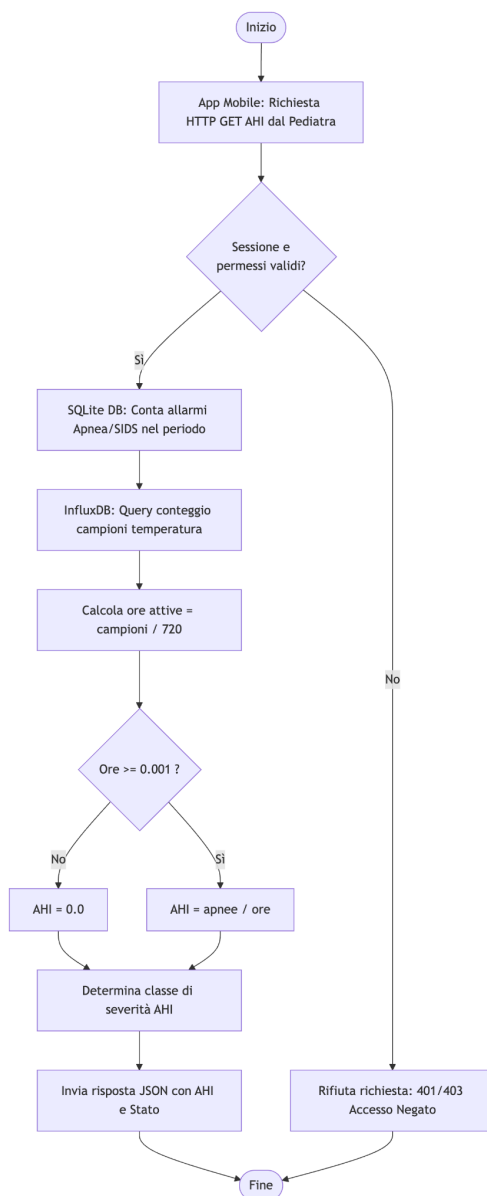
- **Gateway Edge (ESP32):** acquisisce, linearizza e invia i dati vitali e la temperatura NTC al broker.
- **Backend (FastAPI / `mqtt\_handler.py`):** riceve i dati, applica i filtri, verifica il superamento delle soglie cliniche (inclusi calcoli PCA-dipendenti per apnee/bradicardie), decide se generare l'allarme e lo persiste nello storico.
- **Modulo Notifiche / App:** consegna e mostra l'allarme al genitore.

Flusso degli eventi:

1. Inizio: il Gateway riceve le letture filtrate (multi-modali e temperatura NTC).
2. Invio Telemetria: Il Gateway pubblica periodicamente i dati biometrici e di temperatura unificati sul broker MQTT.*
3. Ricezione ed Elaborazione: Il Backend FastAPI intercetta i dati, applica i filtri digitali e interroga il database SQLite per ottenere le soglie cliniche personalizzate associate al neonato.*
4. Decisione sulla Soglia (Backend): Il Backend verifica se i valori superano le soglie cliniche (inclusi calcoli PCA-dipendenti per apnee/bradicardie o allarmi di caduta/postura).*
5. Innesco Allarme: Se una soglia è superata, il Backend genera l'evento di allarme, lo persiste nello storico SQLite e invia in tempo reale le notifiche (via WebSocket all'app mobile e via HTTP API a Telegram).

3.2.3 Calcolo dell'Indice AHI

Descrizione: il diagramma di attività illustra il calcolo on-demand dell'indice come aggregazione cross-database.



Attori / Corsie:

- **App (Pediatra):** richiede l'indice per il paziente e la finestra selezionati.
- **API REST / Analitica Clinica:** verifica i permessi e orchestra il calcolo.
- **DB relazionale:** fornisce il conteggio degli eventi anomali.
- **InfluxDB:** fornisce le ore di monitoraggio.

Flusso degli eventi:

1. Inizio: l'App richiede l'AHI; l'API verifica permessi e associazione paziente–pediatra.
2. Decisione di autorizzazione: se non autorizzato, accesso negato e fine.
3. Conteggio eventi: il DB relazionale restituisce il numero di apnee/SIDS nella finestra.
4. Stima ore: InfluxDB restituisce i campioni di temperatura, convertiti in ore di monitoraggio.
5. Decisione sulle ore: se le ore sono trascurabili, l'indice è neutro (guardia) con stato informativo; altrimenti AHI = eventi / ore.
6. Classificazione di severità e restituzione del risultato; l'App lo visualizza. Fine.

4. Design dell'interfaccia utente

Il design dell'interfaccia segue un approccio minimalista, semplice e intuitivo, per garantire un'esperienza d'uso immediata e priva di distrazioni non necessarie. Le interfacce condividono una struttura coerente; contenuti e azioni mostrate sono determinati dal ruolo dell'utente (Genitore o Pediatra), in coerenza con il controllo degli accessi basato sui ruoli.

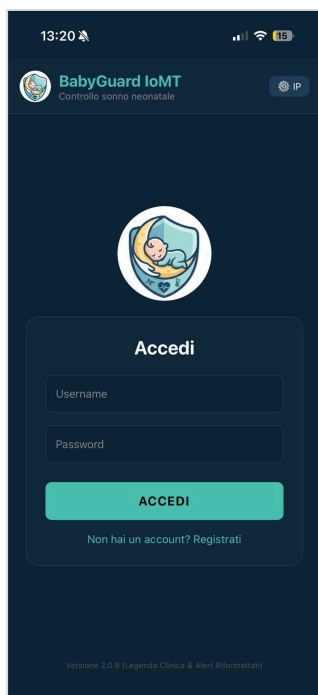


Figura 4.1 Schermata di Login

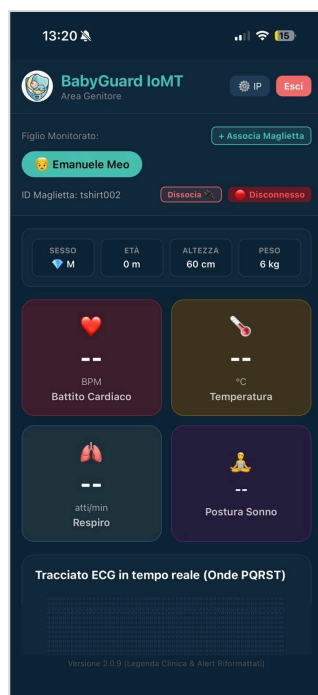


Figura 4.2 Dashboard Genitore



Figura 4.3 Area Pazienti Pediatra

4.1 Schermata di Login

Principali interazioni con l'utente: layout essenziale, focalizzato sull'autenticazione. L'utente interagisce con i campi Username e Password e con il pulsante di accesso. Al click, l'app invia le credenziali alle API REST: in caso di esito positivo è indirizzato alla vista corrispondente al proprio ruolo; in caso contrario viene mostrato un messaggio di alert. La schermata non rivela informazioni sul ruolo finché l'autenticazione non è completata.

4.2 Dashboard Real-time Genitore

Principali interazioni con l'utente: la dashboard del genitore adotta uno stile pulito che evidenzia lo stato corrente del bambino, presentando in un'unica vista riepilogativa i quattro parametri principali aggiornati in tempo reale:

- Frequenza cardiaca (ECG);
- Respiro e indicazione di eventuale apnea (Strain Gauge);
- Posizione, con evidenza chiara della posizione prona (IMU);
- Temperatura cutanea, derivata dalla lettura locale del sensore NTC dell'ESP32.

Un'area di stato comunica in modo immediato la condizione del bambino (normale/allarme) e l'eventuale notifica di batteria scarica o disconnessione. Il genitore visualizza esclusivamente il dispositivo associato al proprio figlio. La dashboard ospita un box informativo espandibile a comparsa (collassabile) che fornisce spiegazioni semplificate dei termini clinici (SIDS, ALTE,

Apnea, Bradicardia, Ipotonia, Ipertermia/Ipotermia) per assistere il genitore nella decodifica delle anomalie.

4.3 Area Pazienti Pediatra

Principali interazioni con l'utente: mantenendo la coerenza stilistica con le altre schermate, l'area del pediatra adotta uno stile pulito per facilitare la navigazione. Le interazioni che la caratterizzano sono:

- consultazione della lista completa dei propri pazienti, con ricerca e selezione;
- selezione di un paziente per accedere ai dati real-time, allo storico essenziale e ai report;
- **consultazione dell'Indice AHI** del paziente con la relativa classe di severità (Normale/Lieve/Moderato/Severo), da interpretare in funzione dell'età postmestruale;
- accesso alle dashboard cliniche di Grafana per l'analisi storica approfondita;
- configurazione delle soglie dei sensori per il paziente selezionato.

Un pulsante di Logout consente di terminare la sessione e tornare alla schermata di Login. L'accesso è limitato ai soli pazienti associati al pediatra autenticato.

Appendice A — Tracciabilità delle modifiche verso i requisiti

La tabella collega ciascuna modifica di design ai requisiti dell'SRS aggiornato e ai casi d'uso interessati, a garanzia della coerenza tra documento di design, requisiti e codice del prototipo.

Modifica di design	Requisiti SRS	Casi d'uso	Sezioni del DDD
Temperatura cutanea acquisita localmente via NTC (rimozione dalla smart shirt e dal canale BLE)	SE-1.1, SE-1.2, FC-1.6	UC-3, UC-6	1.1.1, 1.1.2, 1.1.3, 1.1.4
Topic MQTT dedicato per la temperatura NTC	SE-1.3	UC-6	1.1.5, 1.3.2
Scrittura della temperatura NTC su InfluxDB (vitals_summary)	SE-1.4, DF-1.4	UC-3, UC-6	1.1.6, 1.1.7, 1.2.1
Valutazione delle soglie termiche basata sul dato NTC	SE-1.2, IF-1.7	UC-3, UC-5	1.1.3, 3.1.2, 3.2.2
Calcolo dell'Indice AHI (eventi anomali / ore di sonno)	DF-1.6	UC-4	1.1.10, 1.2.3, 3.1.3, 3.2.3
Storico allarmi come fonte del numeratore AHI	DF-1.6	UC-3, UC-4	1.1.8, 1.2.3
Severità AHI contestualizzata sull'età postmestruale (autocalibrazione soglie)	IF-1.6	UC-5	1.1.10, 4.3
Visualizzazione dell'AHI nell'area clinica del pediatra	UI-1.3, DF-1.6	UC-4	1.1.12, 4.3
Integrazione del Bot Telegram per notifiche di allerta in mobilità	IF-1.11, SE-1.7	UC-7, UC-3	1.1.13, 3.1.2
Glossario e legenda clinica interattiva per i genitori	UI-1.7	UC-3	1.1.12, 4.2